Politechnika Warszawska

WYDZIAŁ MECHATRONIKI



Praca dyplomowa magisterska

na kierunku Automatyka, Robotyka i Informatyka Przemysłowa

Projekt i implementacja schedulera systemów operacyjnych o złożoności O(1)

Kajetan Kontek

numer albumu 327749

promotor

dr inż. Bartłomiej Fajdek

WARSZAWA 2027

Streszczenie

Modułowy szablon pracy dyplomowej w systemie LaTeX

Wytyczne Rektora Politechniki Warszawskiej precyzują kilka rygorystycznych wymogów odnośnie streszczenia pracy dyplomowej. Streszczenie stanowi odrębny punkt w układzie pracy i musi znajdować się na **osobnej stronie**. Obowiązuje tu również **ścisła objętość i formatowanie**, ponieważ każde streszczenie, niezależnie od języka, musi zajmować dokładnie jedną stronę, przy czym należy w nim zastosować pojedynczy odstęp (interlinię) oraz czcionkę o rozmiarze 12 pkt.

Jeśli chodzi o **wymagane elementy**, to poza samym tekstem podsumowującym pracę, w każdym streszczeniu obowiązkowo musi pojawić się tytuł pracy oraz zestaw słów kluczowych. W praktyce akademickiej jako zestaw słów kluczowych przyjmuje się zazwyczaj od 3 do 5 pojęć lub krótkich fraz, które najdokładniej oddają tematykę i zakres przeprowadzonych w pracy badań. Umieszcza się je pod tekstem streszczenia.

Same **wersje językowe** zależą bezpośrednio od tego, w jakim języku napisana jest cała praca. Dla pracy w języku polskim wymagane jest osobne streszczenie po polsku oraz drugie streszczenie po angielsku. Dla pracy w języku angielskim wymagane jest streszczenie po angielsku oraz streszczenie po polsku (w tej kolejności). Z kolei dla pracy w innym języku obcym wymagane jest streszczenie w języku, w którym napisano pracę, a ponadto streszczenie po polsku oraz streszczenie po angielsku. **Opcjonalnie**, jeśli zamierzasz wystąpić o wydanie odpisu dyplomu w tłumaczeniu na jeszcze inny język obcy, powinieneś dołączyć dodatkowe streszczenie napisane w tym właśnie języku.

Warto zadbać o to, aby styl pisania był zwięzły i odzwierciedlał strukturę pracy: cel, metody, wyniki i wnioski.

Słowa kluczowe: praca dyplomowa, układ modułowy, LaTeX

Abstract

Modular diploma thesis template in LaTeX system

Here goes the English version of your abstract. It must follow the exact same formatting rules: 12pt font size, single line spacing, and it must fit entirely on this single page.

Keywords: diploma thesis, modular layout, LaTeX

Spis treści

1 Istniejące algorytmy planowania czasu procesora w nowoczesnych systemach 9
1.1 System Linux i architektura EEVDF 9
1.1.1 Wybór procesu i struktury danych 10
1.1.2 Mechanizm Run to Parity 10
1.2 Systemy z grupy Windows 10
1.2.1 Wybór procesu i struktury danych 11

2 Wymagania edytorskie 13
2.1 Ogólna struktura i formatowanie tekstu 13
2.2 Wypunktowania 13
2.3 Spójniki na końcu linii 13
2.4 Wzory matematyczne 14
2.5 Tabele 17
2.6 Rysunki i grafiki 19
2.7 Kody źródłowe 25
2.8 Bibliografia 26

3 Projekt i realizacja systemu 29

4 Wyniki badań 30

5 Podsumowanie 31

Literatura 32

Wykaz symboli i skrótów 34

Spis załączników 40

1. Istniejące algorytmy planowania czasu procesora w nowoczesnych systemach

1.1 System Linux i architektura EEVDF

W nowoczesnych wersjach jądra systemu Linux, domyślnym algorytmem planowania (ang. *scheduler*) dla procesów standardowych jest algorytm Earliest Eligible Virtual Deadline First (EEVDF). Zastąpił on wieloletni standard znany jako Completely Fair Scheduler (CFS), rozwiązując fundamentalny problem balansu między sprawiedliwością przydziału czasu a opóźnieniami w obsłudze zadań interaktywnych [**1**], [**38**].

Działanie algorytmu EEVDF opiera się na ścisłych podstawach matematycznych [**15**] i operuje na kilku kluczowych wartościach przypisywanych każdemu zadaniu:

- **Żądany kwant czasu** (*requested time slice / slice*)
- **Wirtualny czas wykonania** (*virtual runtime / vruntime*)
- **Opóźnienie** (*lag*)
- **Priorytet** (*priority / weight*)

Kiedy nowe lub powracające ze stanu uśpienia zadanie trafia do kolejki procesów gotowych, otrzymuje bazowy kwant czasu (*slice*). Wartość ta oparta jest na zmiennych konfiguracyjnych systemu (np. `sysctl_sched_base_slice`) i jest dynamicznie skalowana na podstawie wagi procesu (wynikającej z jego priorytetu *nice*) [**19**], [**26**]. Następnie, gdy zadanie zaczyna być wykonywane przez procesor, jego wirtualny czas wykonania (`vruntime`) stale rośnie. Wartość ta odzwierciedla rzeczywisty czas spędzony na procesorze, znormalizowany przez wagę zadania. Zliczanie tego czasu opiera się na precyzyjnych licznikach sprzętowych procesora (np. w architekturze amd64), jednak sama aktualizacja wirtualnego czasu i struktur danych jest w całości realizowana programowo przez jądro systemu operacyjnego, zazwyczaj podczas obsługi przerwań zegarowych [**18**], [**19**].

Na bazie początkowo przypisanego kwantu czasu oraz `vruntime`, system wylicza wirtualny termin zakończenia (*virtual deadline*). Reprezentuje on moment, do którego proces powinien w idealnych warunkach skonsumować swój przydzielony czas. Algorytm na bieżąco oblicza również opóźnienie (*lag*), będące różnicą pomiędzy czasem, jaki zadanie powinno było otrzymać w idealnym, proporcjonalnym modelu podziału czasu (tzw. *fluid-flow model*), a czasem, który faktycznie wykorzystało. Jeżeli wartość *lag* jest mniejsza od zera, oznacza to, że proces otrzymał więcej czasu procesora niż pozostałe procesy rywalizujące o czas procesora. W takim scenariuszu proces traci kwalifikowalność (*ineligible*) i jest kategorycznie wykluczony z możliwości przydziału sprzętowego do momentu, aż ogólny upływ czasu w systemie sprawi, że jego *lag* ponownie wzrośnie do wartości nieujemnej [**1**], [**15**].

1.1.1 Wybór procesu i struktury danych

Działanie każdego planisty w nowoczesnych systemach operacyjnych opiera się na sprzętowym mechanizmie przerwań czasowych (ang. *timer interrupts*). Ponieważ procesor nie może jednocześnie wykonywać własnego kodu schedulera oraz kodu aplikacji użytkownika, wbudowany kontroler przerwań z dużą częstotliwością wstrzymuje wykonywane zadanie, przekazując sterowanie do jądra systemu [**13**]. Daje to algorytmowi możliwość przeliczenia wskaźników i, w razie potrzeby, wywłaszczenia procesu.

Scheduler EEVDF przechowuje informacje o aktywnych procesach w zrównoważonym drzewie czerwono-czarnym (ang. *Augmented Red-Black Tree*). W przeciwieństwie do naiwnego podejścia, drzewo to nadal sortuje węzły na podstawie wirtualnego czasu wykonania (`vruntime`), co układa zadania od lewej do prawej. Kluczową innowacją jest jednak rozszerzenie (*augmentation*) każdego węzła o dodatkowe pole `min_deadline`, które przechowuje najkrótszy termin zakończenia z danego węzła i wszystkich jego potomków. Dzięki temu struktura działa jednocześnie jako drzewo wyszukiwania binarnego oraz jako kopiec (min-heap) dla wartości *deadline*. Architektura ta pozwala systemowi w optymalnym czasie $O(\log n)$ odrzucić całe poddrzewa i natychmiast odnaleźć zadanie o najwcześniejszym *virtual deadline*, które jednocześnie spełnia warunek kwalifikowalności ($lag \geq 0$) i to ono zostaje wybrane do działania na procesorze [**4**], [**19**].

1.1.2 Mechanizm Run to Parity

Algorytm EEVDF w swojej czystej, teoretycznej postaci zakłada wyidealizowany model maszyny obliczeniowej. W rzeczywistości fizyczne procesory opierają swoją wydajność na hierarchii pamięci podręcznej (Cache L1/L2/L3), która przechowuje dane powiązane z aktualnie wykonywanym procesem. Niezmodyfikowany mechanizm doboru zadań oparty wyłącznie na *deadline'ach* prowadziłby do zjawiska tzw. *over-scheduling* – zbyt częstych zmian aktualnie działającego zadania. Skutkowałoby to znacznymi stratami cykli procesora na ciągłą wymianę linii pamięci podręcznej (*cache miss*) oraz narzutem związanym z przełączaniem kontekstu.

Aby rozwiązać ten problem, w systemie Linux wprowadzono mechanizm *Run to Parity* [**26**]. Zgodnie z tą regułą, proces, który już przejął kontrolę nad jednostką obliczeniową, z reguły nie zostaje wywłaszczony przez inne, standardowe zadania posiadające wcześniejszy *virtual deadline*, dopóki jego własne opóźnienie (*lag*) nie spadnie do zera. Dopiero gdy proces "wyrówna" swój dług wobec systemu (osiągając parytet), zdejmuje się z niego ochronę, pozwalając na przejęcie procesora przez kolejne kwalifikujące się zadanie. Mechanizm ten gwarantuje dłuższe, nieprzerwane odcinki pracy pojedynczego wątku, co maksymalizuje wykorzystanie pamięci podręcznej i zwiększa ogólną przepustowość sprzętu (throughput), nie naruszając jednocześnie długoterminowych matematycznych gwarancji sprawiedliwości algorytmu [**38**].

1.2 Systemy z grupy Windows

W nowoczesnych systemach z rodziny Windows za przydział czasu procesora odpowiada planista (ang. *scheduler*) zaprojektowany pierwotnie na potrzeby architektury Microsoft Windows NT. W przeciwieństwie do algorytmów opartych na sprawiedliwym podziale czasu, rozwiązanie firmy Microsoft bazuje na wywłaszczeniowym planowaniu wielozadaniowym, w którym absolut-

nym fundamentem jest rygorystyczny priorytet [3]. Co istotne, jednostką planowania w systemie Windows nie jest proces, lecz pojedynczy wątek. Proces pełni jedynie rolę kontenera na zasoby (takie jak pamięć czy kontekst zabezpieczeń), podczas gdy o dostęp do jednostki obliczeniowej rywalizują bezpośrednio wątki [4].

Architektura planisty definiuje 32 odrębne poziomy priorytetów (od 0 do 31), które podzielone są na trzy główne kategorie [8]:

- **Poziom 0 (Zero-Page Thread):** Zarezerwowany wyłącznie dla systemowego wątku bezczynności (*idle thread*), który uruchamia się tylko przy braku innych zadań. Zajmuje się on m.in. asynchronicznym zerowaniem wolnych stron pamięci fizycznej.
- **Poziomy 1–15 (Priorytety dynamiczne):** Przydzielane standardowym wątkom aplikacji użytkownika oraz niekrytycznym wątkom jądra. Podlegają one ciągłym, dynamicznym algorytmom modyfikacji (wzmacnianiu i degradacji).
- **Poziomy 16–31 (Priorytety czasu rzeczywistego):** Zarezerwowane dla absolutnie krytycznych operacji (np. przetwarzanie multimediów przy minimalnych opóźnieniach, sterowniki sprzętowe). Wątki na tych poziomach mają sztywny priorytet i nie podlegają dynamicznym zmianom [5].

Przy tworzeniu wątku w paśmie dynamicznym przydzielana jest mu tzw. wartość bazowa priorytetu (*base priority*), obliczana na podstawie klasy priorytetu procesu oraz priorytetu wątku wewnątrz tej klasy. Aby zapewnić wysoką responsywność interaktywną i zapobiegać zbyt długiemu oczekiwaniu zadań (*starvation*), planista stosuje mechanizm dynamicznego wzmacniania priorytetu (*priority boosting*). W systemie istnieje sześć głównych rodzajów zdarzeń, które wyzwalają tymczasowe podbicie priorytetu wątku [1]:

1. Zakończenie asynchronicznej operacji wejścia/wyjścia (I/O completion).
2. Wybudzenie wątku przez obiekty synchronizacji (np. semafory, zdarzenia).
3. Zakończenie oczekiwania na zasoby warstwy wykonawczej (Executive resources).
4. Działanie w ramach procesu pierwszoplanowego (Foreground process).
5. Wybudzenie w celu obsługi zdarzeń interfejsu graficznego (GUI windowing activity).
6. Interwencja usługi *Balance Set Manager*, chroniąca przed głodzeniem poprzez wywindowanie priorytetu zablokowanego wątku do wartości 15 [2].

Po otrzymaniu wzmocnienia, kwant czasu (ang. *quantum*) wątku ulega wyzerowaniu. Kiedy wątek w pełni wykorzysta nowo przydzielony kwant, system stosuje mechanizm degradacji (*priority decay*) – jego priorytet jest zmniejszany o 1. Proces ten powtarza się przy każdym kolejnym wyczerpaniu kwantu, aż wartość spadnie z powrotem do nominalnego priorytetu bazowego [1].

1.2.1 Wybór procesu i struktury danych

Kernel utrzymuje niezależne struktury danych dla każdego procesora logicznego, zgrupowane wewnątrz bloku *Kernel Processor Control Block* (KPRCB) [7]. Serce tej struktury stanowią

kolejki gotowości (*Dispatcher Ready Queues*), zaimplementowane jako tablica 32 dwukierunkowych list powiązanych – po jednej dla każdego poziomu priorytetu. Nowe lub wybudzone wątki dodawane są na koniec listy odpowiadającej ich priorytetowi.

W celu optymalizacji wyboru następnego zadania, dyspozytor unika czasochłonnego iterowania po listach. Utrzymuje on pojedynczą, 32-bitową liczbę, zwaną *Ready Summary*. Każdy bit tej zmiennej powiązany jest z jedną kolejką. Jeśli dana kolejka posiada przynajmniej jeden wątek gotowy do natychmiastowego uruchomienia, odpowiadający jej bit ustawiany jest na wartość 1. Jeżeli kolejka jest pusta, bit przyjmuje wartość 0 [8].

Dzięki takiej architekturze, problem wyboru następnego wątku sprowadza się do odnalezienia najstarszego ustawionego bitu (ang. *most significant set bit*) w zmiennej *Ready Summary*. Jest to operacja matematyczna, której złożoność czasowa wynosi $O(1)$. W architekturach sprzętowych z rodziny x86/amd64 poszukiwanie to realizowane jest za pomocą pojedynczej instrukcji procesora – BSR (*Bit Scan Reverse*). Pozwala to w bardzo krótkim czasie zidentyfikować najwyższą niepustą kolejkę, pobrać z niej pierwszy oczekujący wątek i zainicjować przełączenie kontekstu [6].

2. Wymagania edytorskie

2.1 Ogólna struktura i formatowanie tekstu

Prace dyplomowe realizowane na Politechnice Warszawskiej muszą spełniać ścisłe kryteria redakcyjne, które gwarantują spójność estetyczną i edytorską wszystkich publikacji akademickich. Poniżej przedstawiono zestawienie najważniejszych wymagań technicznych dotyczących układu graficznego oraz formatowania tekstu głównego.

- **Format:** praca obowiązkowo posiada układ dwustronny na formacie A4.
- **Podział na rozdziały:** zastosowano maksymalnie trzy poziomy podrozdziałów. Tytuły głównych rozdziałów (I poziomu) zaczynają się od nowej strony. Tytuły te powinny być zapisane pogrubioną czcionką bezszeryfową w malejących rozmiarach: 14 pkt (I poziom), 13 pkt (II poziom) i 12 pkt (III poziom).
- **Krój pisma i interlinia:** tekst główny pisze się zazwyczaj czcionką bezszeryfową (np. Arial) w rozmiarze 11, z zalecaną interlinią 1,15.
- **Numeracja stron:** jest wymagana i musi znajdować się u dołu po zewnętrznej stronie, z zastosowaniem odbicia lustrzanego dla stron parzystych i nieparzystych.
- **Akapity i wyliczenia:** akapity można formatować stosując wcięcie 0,5 cm lub pozostawić je bez wcięcia, stosując odstęp 4 pkt przed akapitem.

2.2 Wypunktowania

System LaTeX posiada ogromne możliwości formatowania list, jednak zgodnie z obowiązującymi zaleceniami edytorskimi Politechniki Warszawskiej, w pracach dyplomowych dopuszcza się wyłącznie dwa rodzaje znaków wyliczenia:

- standardową „kropkę” (jest to domyślny punktor w otoczeniu `itemize`),
  - myślnik (aby go uzyskać, należy dopisać argument `[--]` bezpośrednio po znaczniku `\item`).

Zatem w tekście głównym należy unikać wypunktowań numerycznych (otoczenie `enumerate`), list opisowych i innych niestandardowych form, chyba że wymaga tego ścisła logika wywodu (np. wypisywanie kolejnych kroków algorytmu).

2.3 Spójniki na końcu linii

Należy zwrócić uwagę na nie pozostawianie pojedynczych liter na końcu linijki tekstu (są to tzw. wiszące spójniki i przyimki). Problem ten dotyczy najczęściej małych liter takich jak: a, i, o, u,

w, z, ale warto pamiętać, że zasada ta obejmuje również wielkie litery rozpoczynające nowe zdania (np. A, I, W, Z), o czym autorzy prac często zapominają.

Aby trwale połączyć taką literę z następującym po niej słowem i wymusić ich ewentualne przeniesienie do nowej linii razem, w kodzie LaTeX zamiast zwykłej spacji należy użyć tzw. twardej spacji (niełamliwej). Wstawia się ją za pomocą znaku tyldy (`~`).

Przykłady zastosowania w kodzie:

- **Zapis niezalecany:** `Napięcie w układzie wynosi...` (litera „w” może nieestetycznie zawisnąć na końcu linii w wygenerowanym pliku PDF).
- **Zapis w pełni poprawny:** `Napięcie w~układzie wynosi...` (litera „w” zawsze przeniesie się do nowej linii razem ze słowem „układzie”).

2.4 Wzory matematyczne

W kwestii edycji wzorów matematycznych w pracy dyplomowej należy kierować się wytycznymi i dobrymi praktykami. Podobnie jak w przypadku elementów graficznych, każda zależność matematyczna musi być ściśle zintegrowana z tekstem głównym i bezpośrednio w nim przywołana za pomocą odpowiedniego odsyłacza numerowanego (polecenie `\eqref{...}`). Wzory nie mogą pojawiać się w dokumencie w sposób przypadkowy ani pozostawać bez komentarza ze strony autora.

- **Brak ścisłych regulacji:** oficjalne wytyczne nie określają szczegółowego sposobu numeracji samych wzorów (w przeciwieństwie do tabel i rysunków, które można numerować ciągle lub w obrębie rozdziałów).
- **Standardowa numeracja (wg szablonu):** równania matematyczne standardowo otrzymują kolejne numery – w praktyce stosuje się numerację ciągłą dla całej pracy.
- **Format zapisu:** numery równań umieszcza się po prawej stronie wzoru w nawiasach okrągłych, np. (1), (2), (3).
- **Wzory nienumerowane:** jeśli dane równanie jest mniej ważne, nie musi być numerowane. W edytorze LaTeX osiąga się to poprzez dodanie gwiazdki do nazwy otoczenia (np. `equation*`).
- **Dozwolona zmiana czcionki:** chociaż do pisania tekstu głównego zalecana jest czcionka bezszeryfowa (np. Arial), to w pracach zawierających bardzo dużo wzorów matematycznych i obliczeń dopuszcza się zastosowanie czcionki szeryfowej, takiej jak Times New Roman.
- **Globalny wykaz symboli i oznaczeń:** wszystkie kluczowe, powtarzające się w pracy symbole matematyczne, wielkości fizyczne oraz specyficzne oznaczenia indeksowe powinny zostać zebrane w zbiorczym zestawieniu (*Wykazie symboli i skrótów*). Podczas gdy pod poszczególnymi wzorami stosuje się definicje lokalne (wprowadzane słowem „gdzie:”), wykaz globalny pozwala czytelnikowi na szybką weryfikację znaczenia danej zmiennej i jej jednostki w dowolnym momencie czytania pracy. Zgodnie ze ścisłym układem pracy dyplomowej, zbiorcze zestawienie wszystkich oznaczeń matematycznych oraz

skrótów stanowi osobną część dokumentu i zamieszcza się je na końcu pracy (zaraz po spisie bibliograficznym, a przed spisem rysunków i tabel).

Podczas opisywania zjawisk fizycznych w tekście, zmienne takie jak napięcie U czy prąd I umieszczamy zawsze w trybie matematycznym. Podstawowe równanie opisujące moc czynną w układzie trójfazowym symetrycznym zdefiniowano wzorem (1):

$$P = \sqrt{3} \cdot U \cdot I \cdot \cos(\varphi), \tag{1}$$

gdzie:

- P – moc czynna układu [W],
- U – napięcie międzyfazowe [V],
- I – prąd przewodowy [A],
- $\cos(\varphi)$ – współczynnik mocy.

Dzięki zastosowaniu etykiet (`label`, wewnątrz środowiska `equation`), jak widać na podstawie równania (1), moc zależy bezpośrednio od charakteru obciążenia. W przypadku konieczności obliczenia energii, korzystamy z całki oznaczonej:

$$E = \int_{t_1}^{t_2} P(t)\, dt. \tag{2}$$

W pracach magisterskich, szczególnie na kierunkach takich jak Mechatronika, wzory matematyczne bardzo często zawierają operatory różniczkowe, ułamki, indeksy iteracyjne oraz symbole przybliżeń numerycznych. Aby zachować czytelność tekstu, należy stosować następujące reguły:

- **Zapis pochodnych:** w systemie LaTeX pochodne można zapisywać na dwa sposoby, zależnie od kontekstu:
  - *Notacja Newtona:* stosowana najczęściej przy funkcjach jednej zmiennej, np. `f'(x)`. W kodzie uzyskuje się ją poprzez zwykły apostrof z klawiatury w otoczeniu matematycznym: `$f'(x)$`.
  - *Notacja Leibniza:* stosowana przy pochodnych cząstkowych lub gdy chcemy wyraźnie wskazać zmienną, po której różniczkujemy. Wykorzystuje się do tego standardowe polecenie ułamka, np. `\frac{df}{dx}`.
- **Indeksy dolne w iteracjach:** w metodach numerycznych kluczowe jest poprawne oznaczanie kolejnych kroków algorytmu. Indeks dolny pojedynczy uzyskujemy znakiem podkreślenia `_`, natomiast indeks składający się z kilku znaków musi być bezwzględnie zamknięty w nawiasach klamrowych, np. `x_{n+1}`.
- **Symbole specyficzne dla metod numerycznych:**
  - **Znak przybliżenia:** w algorytmach numerycznych rzadko mamy do czynienia z idealną równością. Zamiast znaku `=`, do oznaczenia aproksymacji stosuje się symbol `\approx` ($\approx$).

  - **Granice i dążenie:** do zapisu warunków zbieżności algorytmu (np. krok h dążący do zera) używa się komendy `\to` ($\to$), na przykład: `h \to 0`.

- **Interpunkcja matematyczna:** wzór matematyczny, nawet jeśli jest wyśrodkowany i umieszczony w osobnym bloku (`equation`), z punktu widzenia gramatyki jest częścią zdania. Oznacza to, że:
  - jeśli wzór kończy myśl, na jego końcu (wewnątrz otoczenia `equation`, tuż po wzorze) musi znaleźć się **kropka**,
  - jeśli po wzorze następuje definicja użytych symboli (wprowadzana słowem „gdzie:”), na końcu wzoru musi znaleźć się **przecinek**.

Jednym z najpopularniejszych algorytmów numerycznego wyznaczania miejsc zerowych funkcji jest metoda stycznych, znana powszechnie jako metoda Newtona-Raphsona. Rozwiązanie iteracyjne opiera się na wykorzystaniu pierwszej pochodnej funkcji celu. Kolejne przybliżenia pierwiastka wyznacza się ze wzoru:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \tag{3}$$

gdzie:

- x_n – wartość przybliżenia w n-tej iteracji,
- $f(x_n)$ – wartość funkcji w punkcie x_n,
- $f'(x_n)$ – wartość pierwszej pochodnej funkcji w punkcie x_n.

W przypadku, gdy wyznaczenie pochodnej w postaci analitycznej $\frac{df}{dx}$ jest niemożliwe lub zbyt kosztowne obliczeniowo, stosuje się numeryczną aproksymację za pomocą ilorazu różnicowego progresywnego:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \tag{4}$$

przy założeniu, że krok h jest dostatecznie mały ($h \to 0$).

W pracach z zakresu mechatroniki i elektrotechniki często zachodzi potrzeba odwołania się do bazowych praw fizyki, które stanowią fundament opisywanych układów. Doskonałym przykładem jest prawo Ohma, definiujące relację między prądem, napięciem a rezystancją w obwodzie elektrycznym. Zależność tę wyraża się za pomocą następującego równania:

$$I = \frac{U}{R}, \tag{5}$$

gdzie:

- I – natężenie prądu elektrycznego [A],
- U – napięcie przyłożone do końców przewodnika [V],
- R – rezystancja (opór) przewodnika [Ω].

Jak wynika z równania (5), natężenie prądu przepływającego przez idealny przewodnik jest wprost proporcjonalne do napięcia oraz odwrotnie proporcjonalne do jego rezystancji.

2.5 Tabele

Wszystkie tabele w pracy dyplomowej muszą być czytelne i wyśrodkowane. Zgodnie z zasadami edytorskimi, podpis (tytuł) tabeli znajduje się zawsze nad nią. Podobnie jak w przypadku rysunków oraz wzorów matematycznych, każda tabela zamieszczona w dokumencie musi posiadać bezpośrednie odniesienie w tekście głównym (realizowane za pomocą polecenia `\ref{...}`). Niedopuszczalne jest pozostawianie tabel jako samodzielnych, „wiszących" bloków danych – autor jest zobowiązany do omówienia lub przeanalizowania zawartości zestawienia w sąsiadującym akapicie. Poniżej znajdują się wytyczne dotyczące tworzenia i formatowania tabel, zebrane na podstawie zarządzenia Rektora PW [**rektorPW2022**] oraz instrukcji z szablonu LaTeX.

Oficjalne wymogi edytorskie:

- **Tytuł tabeli:** należy go umieścić nad tabelą. Powinien być wyjustowany do lewej strony i zapisany czcionką o kroju bezszeryfowym (np. Arial) w rozmiarze 9 pkt.
- **Źródło tabeli:** jeżeli do tabeli dołączone jest źródło, musi się ono znaleźć pod tabelą. Tak jak tytuł, powinno być wyjustowane do lewej strony i zapisane czcionką bezszeryfową o rozmiarze 9 pkt.
- **Numeracja:** autor ma do wyboru dwa warianty numeracji tabel: może być ona ciągła dla całej pracy (np. Tabela 1, 2, 3...) lub prowadzona niezależnie w poszczególnych rozdziałach (np. Tabela 1.1, 1.2, 2.1...).
- **Spis tabel:** Zgodnie ze ścisłym układem pracy dyplomowej, wykaz wszystkich tabel stanowi osobną część dokumentu i zamieszcza się go na końcu pracy (po spisie bibliograficznym, wykazie symboli i skrótów oraz spisie rysunków). W systemie LaTeX generuje się go całkowicie automatycznie za pomocą polecenia `\listoftables`.

Wskazówki techniczne:

- **Struktura otoczeń:** układ komórek wpisuje się w otoczeniu `tabular`. Aby jednak móc dodać do tabeli wymagany podpis (`\caption`) i znacznik umożliwiający odwoływanie się do niej w tekście, otoczenie `tabular` należy zamknąć w nadrzędnym otoczeniu `table`.
- **Formatowanie kolumn:** liczbę i wyrównanie kolumn deklaruje się w argumencie polecenia `tabular`, używając liter: `l` (dosunięcie do lewej), `r` (dosunięcie do prawej), `c` (centrowanie) lub `p` (styl akapitowy z określoną szerokością). Aby dodać pionowe linie odgradzające kolumny, wstawia się między te litery znak pionowej kreski `|`.
- **Formatowanie wierszy:** poszczególne elementy w wierszu oddziela się znakiem `&`, a sam wiersz kończy podwójnym ukośnikiem `\\`. Poziome linie w tabeli rysuje się za pomocą polecenia `\hline` (pojedyncza linia) lub `\hline\hline` (podwójna linia, przydatna np. do oddzielenia nagłówka).
- **Pozycjonowanie na stronie:** lokalizacją tabeli zarządza się poprzez parametry literowe (np. `h` – tutaj, `b` – dół, `t` – góra) i opcjonalny znak wykrzyknika `!`, który wymusza na kompilatorze utrzymanie tabeli w pożądanym miejscu.

W tabeli 1 zestawiono wyniki eksperymentu zbadane dla różnych wartości obciążenia.

Tabela 1. Zestawienie wyników pomiarów dla badanych punktów pracy

Nr pomiaru	Napięcie U [V]	Prąd I [A]	Moc P [W]
1	230.1	1.25	287.6
2	229.8	2.10	482.5
3	230.5	3.50	806.7
4	228.9	4.80	1098.7

Analizując dane zgromadzone w tabeli 1, można zauważyć, że przy spadku napięcia poniżej nominalnych 230 V, system automatycznie zwiększa pobór prądu.

Choć w nowoczesnej typografii naukowej zaleca się unikanie pionowych linii na rzecz tzw. tabel otwartych (bez bocznych krawędzi), w niektórych przypadkach – na przykład przy bardzo gęstych zestawieniach danych – zastosowanie pełnej siatki okazuje się konieczne. Aby poprawnie wygenerować taką tabelę w systemie LaTeX, należy trzymać się klasycznego podejścia do formatowania:

- **Rysowanie pionowych kresek (znak `|`):** za generowanie pionowych linii oddzielających kolumny odpowiada znak tzw. pionowej kreski, wpisywany z klawiatury jako `|`. Umieszcza się go bezpośrednio w parametrach otoczenia `tabular`. Deklaracja `\begin{tabular}{|l|c|c|}` utworzy trzy kolumny, które będą ze wszystkich stron (oraz między sobą) oddzielone ciągłą, pionową linią.

- **Rezygnacja z pakietu `booktabs`:** używając pionowych kresek, należy bezwzględnie zrezygnować z zaawansowanych linii poziomych (takich jak `\toprule`, `\midrule` czy `\bottomrule`). Pakiet `booktabs` wprowadza bowiem własne, niewidoczne marginesy pionowe, co powoduje zrywanie pionowych kresek i powstawanie nieestetycznych luk w siatce.

- **Klasyczne linie poziome (`\hline`):** do rysowania ciągłych poziomych linii, które idealnie łączą się w rogach z pionowymi kreskami, stosuje się wyłącznie standardowe polecenie `\hline`. Wpisuje się je zawsze po poleceniu przejścia do nowej linii, czyli za podwójnym ukośnikiem `\\`.

W tabeli 2 zestawiono parametry popularnych platform obliczeniowych wykorzystywanych w projektach mechatronicznych i systemach wbudowanych.

Tabela 2. Porównanie parametrów technicznych wybranych platform mikrokontrolerowych

Platforma sprzętowa	Architektura rdzenia	Taktowanie	Pamięć RAM
Arduino Uno Rev3	8-bit AVR	16 MHz	2 KB
ESP32 NodeMCU	32-bit Xtensa	240 MHz	520 KB
STM32 Nucleo-64	32-bit ARM Cortex-M4	84 MHz	96 KB
Raspberry Pi 4B	64-bit ARM Cortex-A72	1,5 GHz	do 8 GB

Źródło: Opracowanie własne na podstawie not aplikacyjnych producentów.

Jak wynika z powyższego zestawienia w tabeli 2, do zadań wymagających zaawansowanego przetwarzania obrazu najlepiej sprawdzi się architektura 64-bitowa.

W uzasadnionych przypadkach – na przykład podczas prezentacji obszernych wyników pomiarów, zestawień parametrów czy macierzy – w pracy dyplomowej dopuszcza się zastosowanie tabel o pełnej siatce, przypominających wyglądem arkusz programu Microsoft Excel. Aby uzyskać profesjonalny i estetyczny efekt w systemie LaTeX, należy kierować się następującymi wytycznymi:

- **Wymagane pakiety:** obsługa kolorowania komórek i wierszy nie jest domyślnie wbudowana w podstawowe środowisko tabel. Wymaga ona załadowania w preambule pakietu `xcolor`.
- **Kolorowanie tła:** do wyróżnienia całego wiersza służy polecenie `\rowcolor{...}`, natomiast pojedynczą komórkę (np. zawierającą kluczowy wynik pomiaru) podświetla się za pomocą `\cellcolor{...}`. Dobrą praktyką edytorską jest stosowanie delikatnych odcieni szarości (np. `gray!10` dla naprzemiennych wierszy oraz `gray!25` dla nagłówka) i pastelowych, niejaskrawych barw (np. `red!25`) do wyróżniania najważniejszych danych.
- **Zmiana koloru siatki:** aby gęsta, pełna siatka linii pionowych i poziomych nie przytłaczała czytelnika i nie odciągała wzroku od samych liczb, warto zmienić jej kolor z domyślnej czerni na jasnoszary. Służy do tego polecenie `\arrayrulecolor{gray!50}`. Należy jednak pamiętać, aby tuż po zamknięciu otoczenia tabeli przywrócić czarny kolor poleceniem `\arrayrulecolor{black}`, co uchroni kolejne tabele przed odziedziczeniem tego formatowania.
- **Przestrzeń w komórkach:** standardowe tabele w LaTeXu są bardzo ciasne w pionie. Aby nadać im styl nowoczesnych arkuszy kalkulacyjnych, zaleca się bezpośrednio przed rozpoczęciem otoczenia tabeli wpisać polecenie `\renewcommand{\arraystretch}{1.3}`, co zwiększy wysokość wszystkich wierszy o 30 procent.

W tabeli 3 przedstawiono zbiorcze wyniki pomiarów sprawności układu napędowego.

Tabela 3. Wyniki pomiarów sprawności napędu

Nr	Punkt pracy pomiaru	Prąd I [A]	Sprawność [%]
1	Minimalne obciążenie wału	1,2	72,4
2	Obciążenie znamionowe niskie	2,5	81,1
3	Optymalny punkt pracy układu	4,2	**89,6**
4	Przeciążenie dopuszczalne	5,8	83,3
5	Stan bliski utykowi (maksymalny)	8,1	64,2

Źródło: Opracowanie własne na podstawie pomiarów laboratoryjnych.

2.6 Rysunki i grafiki

Rysunki, wykresy oraz schematy blokowe stanowią kluczowy element pracy dyplomowej, pozwalający na wizualizację wyników i konstrukcji. Ponieważ materiały te są bezpośrednią prezentacją autorskich osiągnięć, autorskich projektów oraz wykonanych stanowisk badawczych,

kluczowe jest zadbanie o ich nienaganną jakość i wysoką rozdzielczość. Estetyczna, ostra grafika pozwala promotorowi oraz recenzentowi w pełni docenić realny wkład i trud włożony w realizację części praktycznej, podczas gdy rozmyte rysunki czy nieczytelne osie wykresów drastycznie obniżają jakość całej pracy. Należy przy tym pamiętać, że każdy zamieszczony obiekt graficzny musi zostać bezpośrednio omówiony oraz przywołany w tekście głównym za pomocą odpowiedniego odsyłacza. Poniżej znajdują się oficjalne zasady Politechniki Warszawskiej oraz praktyczne wytyczne techniczne systemu LaTeX związane z ich wdrażaniem.

Oficjalne zasady redagowania:

- **Podpis rysunku:** zaleca się umieszczać go pod rysunkiem. Powinien być wyjustowany do lewej strony i zapisany czcionką bezszeryfową w rozmiarze 9 pkt. Podpis pod rysunkiem czy tytuł nad tabelą są traktowane jak tytuły rozdziałów lub nagłówki – a w języku polskim nagłówków nie kończymy kropką.
- **Źródło rysunku:** jeżeli do rysunku dołączone jest źródło, należy je umieścić pod rysunkiem (z reguły pod właściwym podpisem). Obowiązuje tu identyczne formatowanie: tekst wyjustowany do lewej strony, czcionka bezszeryfowa, rozmiar 9 pkt.
- **Numeracja:** autor ma swobodę wyboru – numeracja rysunków może być ciągła dla całej pracy (np. Rysunek 1, 2, 3...) lub prowadzona kolejno w poszczególnych rozdziałach (np. Rysunek 1.1, 1.2, 2.1...).
- **Spis rysunków:** zgodnie ze ścisłym układem pracy dyplomowej, wykaz wszystkich rysunków stanowi osobną część dokumentu i zamieszcza się go na końcu pracy (po spisie bibliograficznym oraz wykazie symboli i skrótów). W systemie LaTeX generuje się go automatycznie poleceniem `\listoffigures`.

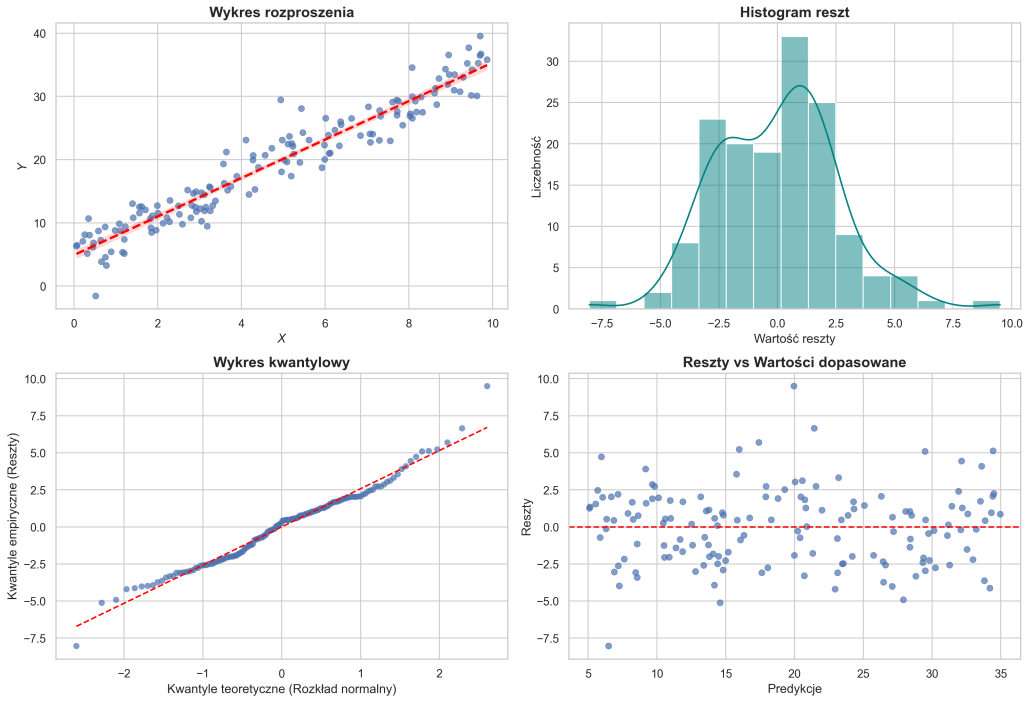
Wskazówki techniczne (na podstawie zaleceń z szablonu):

- **Jakość obrazu:** prace dyplomowe docelowo podlegają archiwizacji cyfrowej. Do schematów, wykresów czy struktur blokowych zdecydowanie poleca się stosowanie grafiki wektorowej (np. formaty PDF lub EPS), która zapewnia idealnie gładkie krawędzie przy każdym powiększeniu. W przypadku tradycyjnych grafik rastrowych (np. zdjęć w formacie PNG lub JPG) kluczowym wymogiem jest zapewnienie wysokiej rozdzielczości (minimalna to 300 DPI), pozwalającej na uniknięcie rozmycia obrazu i utraty szczegółów w finalnym dokumencie.
- **Grafiki rastrowe i zdjęcia:** Wymagają zachowania wysokich parametrów jakościowych, tak aby wszystkie prezentowane na nich elementy były w pełni wyraźne i łatwe do zinterpretowania.
- **Wymiary i spójność:** dokument zachowuje spójność wizualną, gdy szerokości rysunków są powtarzalne. Dobrą praktyką jest zdefiniowanie 2 lub 3 standardowych szerokości relatywnych, np. `0.5\textwidth` dla małych schematów lub `0.8\textwidth` dla rozbudowanych wykresów.
- **Umiejscowienie w tekście:** rysunek musi znajdować się fizycznie w obrębie tego samego rozdziału, w którym został opisany. W systemie LaTeX realizuje się to poprzez zamknięcie grafiki w otoczeniu `figure` i dodanie unikalnego znacznika `\label{...}`.

Pozwala to na automatyczne odwoływanie się do obiektu w tekście za pomocą polecenia `\ref{...}`.

W celu weryfikacji poprawności dopasowania modelu regresji liniowej oraz sprawdzenia kluczowych założeń metody najmniejszych kwadratów (MNK), przeprowadzono pełną analizę diagnostyczną wygenerowanych reszt. Zbiorcze wyniki tej procedury przedstawiono na rysunku 1.

Wykres rozproszenia (lewy górny panel rysunku 1) przedstawia liniową zależność zmiennej objaśnianej Y od zmiennej objaśniającej X wraz z dopasowaną prostą regresji (zaznaczoną czerwoną linią przerywaną). Układ punktów potwierdza silny trend liniowy w badanej próbie. Kolejnym etapem była ocena normalności rozkładu składnika losowego (reszt). Służą do tego dwa kolejne wykresy diagnostyczne: histogram reszt (prawy górny panel) oraz wykres kwantylowy Q-Q (lewy dolny panel). Kształt histogramu przypomina klasyczną krzywą Gaussa, natomiast na wykresie kwantylowym punkty empiryczne niemal idealnie pokrywają się z teoretyczną linią referencyjną. Oba te testy wizualne jednoznacznie potwierdzają, że reszty mają w przybliżeniu rozkład normalny. Ostatni z wykresów – zależność reszt od wartości dopasowanych (prawy dolny panel) – pozwala na ocenę homoskedastyczności, czyli stałości wariancji błędu. Równomierne, losowe rozproszenie punktów wokół czerwonej linii zero ($e = 0$) świadczy o braku struktur nieliniowych oraz o stałej wariancji składnika losowego w całym zakresie predykcji.



Rysunek 1. Wyniki analizy regresji liniowej oraz wykresy diagnostyczne reszt modelu

Źródło: Opracowanie własne na podstawie symulacji numerycznej w środowisku Python.

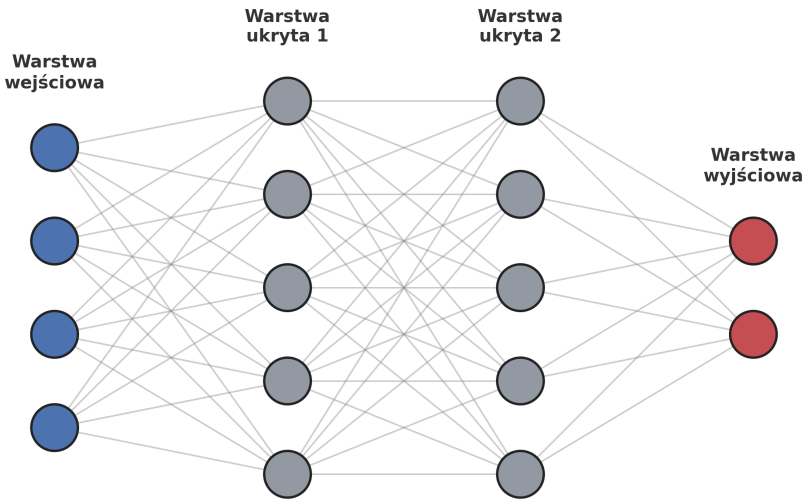
W pracach dyplomowych niezwykle często pojawiają się schematy struktur algorytmów, sieci neuronowych czy systemów sterowania. Poza ogólnymi zasadami umieszczania grafik, w przypadku tego typu schematów obowiązują dedykowane kryteria edytorskie i analityczne:

- **Logika opisu:** opisując schemat strukturalny w tekście pracy, należy zachować naturalny kierunek przepływu sygnałów lub informacji – najczęściej od lewej (wejścia) do prawej

(wyjścia). Tekst powinien przeprowadzić czytelnika przez kolejne etapy przetwarzania danych.

- **Czytelność fontów wewnętrznych:** generując wykresy lub schematy za pomocą kodu (np. w bibliotece `Matplotlib`), należy upewnić się, że napisy na samym rysunku (np. nazwy warstw, opisy osi) po wstawieniu do LaTeXa będą miały rozmiar zbliżony do tekstu głównego lub przypisów. Zbyt małe, nieczytelne litery na rysunku to błąd estetyczny.
- **Autonomiczność obiektu graficznego:** rysunek wraz z podpisem musi stanowić zrozumiałą całość. Niedopuszczalne jest wklejenie schematu sieci (np. neuronowej) bez wyjaśnienia, co oznaczają poszczególne kolory, bloki czy liczby neuronów (zarówno w tekście głównym, jak i skrótowo w podpisie).

W celu realizacji zadania klasyfikacji stanów pracy układu mechatronicznego zaprojektowano sztuczną sieć neuronową typu MLP (ang. *Multi-Layer Perceptron*). Strukturę topologiczną zaimplementowanego modelu przedstawiono na rysunku 2.



Rysunek 2. Struktura sieci neuronowej z dwiema warstwami ukrytymi

Źródło: Opracowanie własne przy użyciu biblioteki Matplotlib w środowisku Python.

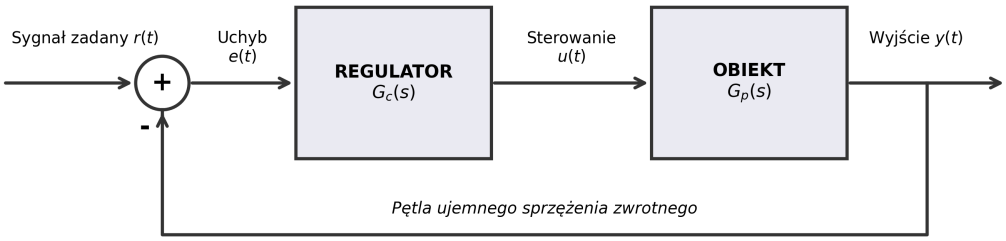
Omawiany model składa się z trzech głównych segmentów: warstwy wejściowej zawierającej 4 neurony (odpowiadające parametrom wejściowym systemu), dwóch warstw ukrytych (po 5 neuronów w każdej) odpowiedzialnych za ekstrakcję cech nieliniowych, oraz dwu–neuronowej warstwy wyjściowej generującej ostateczną decyzję klasyfikatora. Połączenia synaptyczne realizowane są w architekturze każdy z każdym (ang. *Fully Connected*), co pozwala na pełną propagację sygnałów w kierunku do przodu. Szary kolor warstw ukrytych na rysunku 2 oznacza, że ich stany pośrednie nie są bezpośrednio widoczne na wyjściu systemu, a proces ich uczenia przebiegał z użyciem algorytmu wstecznej propagacji błędów.

W pracach o charakterze inżynierskim na schematach blokowych 3, wykresach czy architekturach sieciowych bardzo często pojawiają się oznaczenia matematyczne (np. symbole wielkości fizycznych, zmienne czasowe, transmitancje operatorowe). Absolutnym wymogiem wysokiej

jakości edytorskiej jest zachowanie pełnej spójności wizualnej i merytorycznej między grafiką a tekstem rozprawy:

- **Identyczny krój czcionki:** jeśli na rysunku dany symbol został wygenerowany jako kursywa matematyczna (np. $r(t)$, $G_c(s)$, y), to w tekście głównym ten sam symbol **musi bezwzględnie** zostać otoczony znakami dolara: `$r(t)$`, `$G_c(s)$`, `$y$`. Niedopuszczalne jest stosowanie zwykłego tekstu (np. pisanie „r(t)” zamiast „$r(t)$”), ponieważ LaTeX używa zupełnie innych fontów do tekstu i do matematyki.
- **Jednolitość indeksów:** szczególną uwagę należy zwrócić na indeksy dolne i górne. Jeśli na schemacie blokowym widnieje oznaczenie X_{max} lub G_{p1}, w tekście oraz we wzorach niedopuszczalne jest stosowanie zapisów typu $Xmax$ czy G_{P1} (zmieniona wielkość litery w indeksie).
- **Obowiązek pełnej definicji:** każdy pojedynczy symbol matematyczny lub fizyczny, który został umieszczony na schemacie, musi zostać jawnie zdefiniowany i wyjaśniony w akapicie opisującym ten rysunek. Czytelnik nie może samodzielnie domyślać się znaczenia zmiennych.

Podstawowym elementem projektowanego systemu jest układ regulacji automatycznej, którego schemat blokowy przedstawiono na rysunku 3.



Rysunek 3. Schemat blokowy zamkniętego układu regulacji automatycznej z ujemnym sprzężeniem zwrotnym
Źródło: Opracowanie własne w środowisku Python przy użyciu pakietu Matplotlib.

Działanie układu opiera się na ciągłym pomiarze sygnału wyjściowego $y(t)$, który poprzez pętlę sprzężenia zwrotnego trafia do węzła sumacyjnego. Następuje tam porównanie wartości rzeczywistej z sygnałem zadanym $r(t)$. Wynikiem tego działania jest uchyb regulacji $e(t)$, definiowany zależnością:

$$e(t) = r(t) - y(t). \tag{6}$$

Uchyb ten podawany jest na wejście bloku regulatora o transmitancji operatorowej $G_c(s)$, którego zadaniem jest wygenerowanie odpowiedniego sygnału sterującego $u(t)$ dla obiektu $G_p(s)$.

Przy zmianie rozmiaru grafiki w kodzie (`width`) nigdy nie należy zmieniać ręcznie wysokości w sposób, który zniekształciłby lub rozciągnął obraz. Jeżeli rysunek jest kopią, przerysowaniem lub adaptacją, na końcu podpisu należy bezwzględnie podać źródło za pomocą polecenia `\cite`. Każdy rysunek musi posiadać swoje odwołanie w tekście go poprzedzającym (np. *„...co przedstawiono na rys. \ref{fig:quadrocopter_lot}”*).

Współczesne podejście do syntezy zaawansowanych algorytmów sterowania dla obiektów latających typu Quadrokopter zostało szczegółowo przedstawione w pracy [**bonislawski2014projekt**]. Na rysunku 4 pokazano quadrocopter w czasie lotu.



Rysunek 4. Quadrocopter w trakcie lotu

Źródło: [**bonislawski2014projekt**].

W przypadku wykorzystania w pracy dyplomowej materiałów graficznych wygenerowanych przez sztuczną inteligencję (AI), kluczową kwestią jest zachowanie pełnej przejrzystości. Każdy taki rysunek musi zostać jednoznacznie oznaczony, aby czytelnik oraz recenzent nie zostali wprowadzeni w błąd, że jest to autorska fotografia rzeczywistego obiektu laboratoryjnego lub tradycyjny schemat inżynierski.

Wszelkie ilustracje tego typu powinny być traktowane jako specyficzna forma syntezy wizualnej. Powszechnie akceptowaną formą opisu jest wskazanie w źródle, że grafika stanowi opracowanie własne z jawnym określeniem użytego narzędzia (np. Gemini).



Rysunek 5. Wizualizacja koncepcyjna quadrocoptera w trakcie lotu

Źródło: Opracowanie własne przy użyciu narzędzia AI (Gemini).

2.7 Kody źródłowe

W interdyscyplinarnych projektach realizowanych na Wydziale Mechatroniki, integralną częścią pracy dyplomowej jest bardzo często warstwa aplikacyjna – obejmująca implementację algorytmów sterowania, oprogramowania systemów wbudowanych czy skryptów analizy danych. Wklejanie kodu źródłowego bezpośrednio w strukturę zwykłego tekstu drastycznie obniża czytelność dokumentacji i jest uznawane za błąd edytorski.

Warto podkreślić, że oficjalne Zarządzenie Rektora Politechniki Warszawskiej w sprawie zasad przygotowywania prac dyplomowych [**rektorPW2022**] nie definiuje szczegółowych wymagań ani specyficznych regulacji dotyczących sposobu formatowania kodów programistycznych. Brak sztywnych wytycznych nie zwalnia jednak autora z dbałości o jakość techniczną pracy. W tym zakresie należy bezwzględnie kierować się dobrymi praktykami inżynierskimi oraz standardami publikacyjnymi, których celem jest zapewnienie maksymalnej przejrzystości, estetyki i czytelności prezentowanych struktur algorytmicznych.

Do profesjonalnego i zautomatyzowanego składu kodów w systemie LaTeX standardowym pakietem jest `listings`. Wymusza on poprawne formatowanie techniczne, izoluje kody od narracji głównej oraz ułatwia analizę logiki programistycznej. Poniżej zestawiono kluczowe zasady, którymi należy się kierować podczas integracji fragmentów oprogramowania z tekstem pracy:

- **Składnia i czytelność:** pakiet automatycznie rozpoznaje słowa kluczowe, komentarze i ciągi znaków dla wielu języków programowania (np. C++, Python, MATLAB, Java). Dobrą praktyką jest włączenie numeracji linii po lewej stronie, co znacząco ułatwia odwoływanie się do konkretnych instrukcji w tekście głównym pracy.
- **Podpisy i numeracja:** fragmenty kodu traktuje się jako niezależne byty edytorskie – obok tabel i rysunków. Powinny one posiadać własny, numerowany podpis (z reguły umieszczany nad blokiem kodu).
- **Sposób dołączania i archiwizacji kodu:** krótkie fragmenty (do kilkunastu linii) o charakterze ilustracyjnym zamieszcza się bezpośrednio w treści rozdziałów, zamykając je w otoczeniu `\begin{lstlisting}` ...`\end{lstlisting}`. W przypadku, gdy kod źródłowy programu jest obszerny i jego obecność w tekście głównym zaburzałaby czytelność głównych rozdziałów, zgodnie z zatwierdzoną przez Rektora strukturą dokumentu, można umieścić go na samym końcu pracy jako Załącznik. Fakt ten wymaga formalnego odnotowania w *Wykazie załączników*, umieszczonym w dokumencie. Z kolei kompletne kody źródłowe, całe repozytoria, bazy danych lub duże modele obliczeniowe **należy bezwzględnie wgrać do uczelnianego systemu USOS APD jako oficjalny załącznik cyfrowy** (w formacie skompresowanym, np. `.zip`) do pracy dyplomowej. Wszelkie techniczne i specyficzne wymagania w tym zakresie dyplomant powinien każdorazowo skonsultować z promotorem pracy.

W symulacjach numerycznych kluczowe znaczenie ma poprawna oraz przejrzysta implementacja algorytmów obliczeniowych. Jako wzór poprawnego sformatowania i osadzenia kodu w strukturze tekstu pracy dyplomowej, kod źródłowy 1 przedstawia autorską implementację metody trapezów, służącej do numerycznego całkowania funkcji jednej zmiennej.

Kod źródłowy 1. Implementacja numerycznego całkowania metodą trapezów

```python
import numpy as np

def metoda_trapezow(f, a, b, n):
        # Numeryczne obliczanie łcaki oznaczonej funkcji f(x)
        # w przedziale [a, b] przy żuyciu n łpodprzedziaów.

        x = np.linspace(a, b, n + 1)
        y = f(x)
        h = (b - a) / n

        # Obliczenie świartoci calki zgodnie z łąformu kwadratury
        calka = h * (0.5 * y[0] + np.sum(y[1:-1]) + 0.5 * y[-1])
        return calka
```

Zaprezentowany kod źródłowy 1 wykorzystuje zoptymalizowaną bibliotekę `NumPy` do efektywnego operowania na wektorach danych bez konieczności stosowania pętli iteracyjnych `for`. Funkcja `np.linspace` służy do równomiernego podziału dziedziny całkowania na n podprzedziałów o jednostkowej szerokości kroków h.

Ostateczny wynik obliczeń jest wyznaczany poprzez zsumowanie wartości funkcji na krańcach przedziałów ($y[0]$ oraz $y[-1]$) pomnożonych przez wagę $0,5$ oraz wartości wewnętrznych z wagą $1,0$. Podejście to bezpośrednio odzwierciedla matematyczne sformułowanie metody trapezów. Pełny skrypt zawierający funkcje testowe oraz procedurę analizy błędów został przedstawiony w załączniku 1.

2.8 Bibliografia

Zgodnie z oficjalnymi wytycznymi Rektora Politechniki Warszawskiej [**rektorPW2022**] oraz założeń niniejszego szablonu, formatowanie bibliografii oraz odwołań literaturowych opiera się na kilku kluczowych zasadach podzielonych na obszar wymogów formalnych oraz wskazówek technicznych.

Oficjalne wymogi edytorskie:

- **Sposób cytowania:** dyplomant ma do wyboru dwa dopuszczalne systemy oznaczania źródeł: styl numeryczny (zgodny z polską normą, gdzie odwołania mają postać np. [**wroblewski2023**]) lub styl harwardzki (w którym odwołania przyjmują postać nazwiska autora oraz roku wydania publikacji).
- **Kolejność w spisie końcowym:** bezwzględnym wymogiem uczelnianym jest, aby wykaz literatury umieszczony na końcu pracy był ułożony w porządku alfabetycznym według nazwisk autorów (a nie według kolejności cytowania w tekście).

Wskazówki techniczne dla systemu LaTeX:

- **Pełna automatyzacja:** zarządzanie bibliografią w szablonie realizowane jest automatycznie. Wszystkie noty bibliograficzne (dane bibliograficzne książek, artykułów naukowych czy dokumentacji technicznych) przechowywane są w zewnętrznym pliku bazy danych z rozszerzeniem `.bib` (np. `thesis-references.bib`).

- **Mechanizm odwołań:** do przywoływania konkretnych źródeł w tekście służy polecenie `\cite{znacznik}`. System umożliwia jednoczesne cytowanie kilku pozycji (np. `\cite{znacznik1, znacznik2}`) oraz wskazywanie konkretnych jednostek redakcyjnych, np. stron lub rozdziałów (`\cite[s.~38]{znacznik}`).
- **Eliminacja błędów typograficznych:** aby uniknąć pozostawienia odnośnika bibliograficznego na początku nowej linii, przed poleceniem cytowania należy bezwzględnie stosować twardą spację, wpisując w kodzie: `~\cite{znacznik}`.
- **Dynamiczna zawartość spisu:** do końcowego wykazu literatury trafiają wyłącznie te pozycje, do których autor faktycznie odwołał się w treści pracy. Pozwala to na gromadzenie w pliku `.bib` szerokiej bazy źródłowej, bez ryzyka sztucznego i niepoprawnego zawyżania spisu końcowego.
- **Pozorny nieład numeryczny:** ponieważ bibliografia automatycznie sortuje alfabetycznie pozycje w spisie końcowym, w samym tekście numery odwołań mogą pojawiać się w sposób nieliniowy i niemonotoniczny (np. w danym akapicie pojawi się odwołanie `[5]`, a w kolejnym `[2]`). Jest to zjawisko całkowicie prawidłowe i nie należy go ręcznie modyfikować.

W oficjalnym układzie pracy dyplomowej, rozdział *Bibliografia* (w szablonie generowany automatycznie komendą `\printbibliography` pod nazwą *Literatura*) umiejscowiony jest bezpośrednio po głównych rozdziałach merytorycznych i podsumowaniu. Zastosowanie nagłówka „Literatura” zamiast domyślnego określenia „Bibliografia” jest opcjonalnym zabiegiem, pozwalającym na elastyczne dostosowanie nazewnictwa spisu. Tuż po nim (z odstępem pustej strony), należy umieścić *Wykaz symboli i skrótów*.

Należy pamiętać, że **przed przystąpieniem do generowania spisu literatury, należy bezwzględnie zmienić domyślne narzędzie** *BibTeX* w ustawieniach swojego edytora na **Biber**.

Przykłady prawidłowego odwoływania się do literatury i źródeł w treści pracy:

1. **Odniesienie ogólne (synteza literatury na końcu zdania):**
   „W literaturze podkreśla się, że nowoczesne układy wymagają podejścia interdyscyplinarnego [**wroblewski2023**]”.

2. **Odwołanie bezpośrednie do konkretnej pracy oraz dokumentacji:**
   „Zaimplementowany algorytm bazuje na rozwiązaniu opisanym w pracy [**kowalski2025**], natomiast pełną specyfikację mikrokontrolera zaczerpnięto z dokumentacji technicznej [**stmicro2024**]”.

3. **Przywołanie podstaw teoretycznych i monografii:**
   „Podstawy teoretyczne dotyczące zastosowania algorytmów sztucznej inteligencji w przemysłowych systemach kontroli jakości zaczerpnięto z opracowania [**zielinski2024vision**]”.

4. **Cytowanie dokumentu patentowego:**
   „Podczas projektowania układu jezdnego robota mobilnego inspirowano się rozwiązaniem konstrukcyjnym o zwiększonej przyczepności, które zostało zastrzeżone w patencie [**kowalskiPatent2024**]”.

5. **Odesłanie do szczegółowego omówienia danej metody:**
   „Współczesne podejście do syntezy zaawansowanych algorytmów sterowania dla obiektów latających typu Quadrokopter zostało szczegółowo przedstawione w pracy [**nowak2025control**]”.

3. Projekt i realizacja systemu

Niniejszy rozdział stanowi zasadniczą część merytoryczną pracy dyplomowej. Autor szczegółowo opisuje w nim strukturę projektową urządzeń, układów lub algorytmów oraz ich praktyczną realizację.

Treść powinna w sposób ustrukturyzowany dokumentować proces realizacji projektu od założeń wstępnych do efektu końcowego, uwzględniając:

- opracowanie koncepcji oraz uzasadnienie wyboru stosowanych metod, narzędzi, technologii lub elementów składowych,
- dokumentację projektową i inżynierską (np. schematy, modele matematyczne, diagramy blokowe, algorytmy),
- przebieg prac implementacyjnych, realizacyjnych lub symulacyjnych oraz integrację poszczególnych modułów i podsystemów.

Stylistyka: tekst należy redagować w bezosobowej formie czasu przeszłego (np. *„zaprojektowano”, „zaimplementowano”, „przeprowadzono dobór”*), zachowując charakter sprawozdania technicznego.

4. Wyniki badań

Niniejszy rozdział służy prezentacji, eksperymentalnej walidacji oraz krytycznej analizie uzyskanych wyników badawczych, pomiarowych lub symulacyjnych.

Treść powinna w sposób obiektywny dokumentować działanie i charakterystykę opracowanego rozwiązania, uwzględniając:

- opis metodyki badawczej, warunków przeprowadzenia eksperymentu lub scenariuszy testowych,
- opracowane i czytelnie sformatowane dane wynikowe zaprezentowane w formie tabel zbiorczych oraz wykresów,
- interpretację uzyskanych zależności, analizę błędów pomiarowych oraz odniesienie wyników do założeń początkowych i stanu wiedzy.

Stylistyka: w tym rozdziale nie należy używać określeń subiektywnych (np. *„wyniki są bardzo dobre”, „system działa świetnie”*). Należy posługiwać się wyłącznie językiem faktów, liczb i konkretnych wskaźników inżynierskich, stosując bezosobową formę czasu przeszłego.

5. Podsumowanie

Niniejszy rozdział stanowi formalne zamknięcie merytorycznej części pracy dyplomowej. Jego celem jest zwięzła synteza uzyskanych rezultatów oraz globalna ocena stopnia realizacji zadania inżynierskiego.

W rozdziale tym należy dokonać całościowej oceny zrealizowanych prac inżynierskich, opierając strukturę wniosków na następujących kluczowych aspektach:

- jednoznacznej weryfikacji stopnia realizacji celów oraz tez sformułowanych we wstępie pracy,
- zwięzłym omówieniu najważniejszych osiągnięć o charakterze konstrukcyjnym, programistycznym bądź eksperymentalnym,
- obiektywnej ocenie ograniczeń wypracowanego rozwiązania oraz wskazaniu potencjalnych kierunków jego dalszej optymalizacji lub rozbudowy.

Stylistyka: wnioski powinny być sformułowane w sposób konkretny i zwięzły. Należy unikać dosłownego powtarzania zdań z poprzednich rozdziałów, skupiając się na ogólnej konkluzji z całej pracy.

Więcej informacji na temat systemu LaTeX:

- https://www.overleaf.com/learn – bardzo przystępny, oficjalny tutorial i baza wiedzy na stronie platformy Overleaf,
- https://www.latex-project.org/ – oficjalna strona domowa projektu LaTeX, zawierająca aktualności i dokumentację standardów,
- https://www.tug.org/begin.html – prowadzony przez *TeX Users Group* obszerny zbiór odnośników do materiałów dla osób początkujących,
- https://www.texstudio.org/ – oficjalna strona domowa edytora TeXstudio, z której należy pobierać najnowsze, stabilne wersje programu,
- https://www.xm1math.net/texmaker/ – oficjalna strona domowa edytora Texmaker, z której można pobrać program oraz uzyskać dostęp do jego podręcznika użytkownika,
- https://texstudio-org.github.io/ – oficjalny, szczegółowy podręcznik użytkownika (*User Manual*) wyjaśniający zaawansowane funkcje edytora TeXstudio,
- https://tex.stackexchange.com/ – największe na świecie forum społecznościowe typu Q&A (pytania i odpowiedzi), na którym można znaleźć rozwiązanie niemal każdego problemu technicznego i błędu kompilacji.

Literatura

[1] BSODTutorials. *Thread Quantum, Thread Priority Boosting and Processor Affinity*. 2013. URL: http://bsodtutorials.blogspot.com/2013/11/thread-quantum-thread-priority-boosting.html (term. wiz. 11. 10. 2026).

[2] Joe Duffy. *Priority-induced starvation: Why Sleep(1) is better than Sleep(0) and the Windows balance set manager*. 2006. URL: https://joeduffyblog.com/2006/08/22/priorityinduced-starvation-why-sleep1-is-better-than-sleep0-and-the-windows-balance-set-manager/ (term. wiz. 11. 10. 2026).

[3] Loyola University Chicago, Department of Computer Science. *Process/Thread Scheduling – Introduction to Operating Systems: Principles and Practice*. 2026. URL: https://os.cs.luc.edu/scheduling.html (term. wiz. 11. 10. 2026).

[4] Microsoft. *About Processes and Threads*. 2025. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads (term. wiz. 11. 10. 2026).

[5] Microsoft. *Priority Boosts*. 2025. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/priority-boosts (term. wiz. 11. 10. 2026).

[6] ReactOS Project. *ntoskrnl/include/internal/ke_x.h File Reference*. ReactOS 0.4.17-dev. 2026. URL: https://doxygen.reactos.org/de/d90/ke__x_8h.html (term. wiz. 11. 10. 2026).

[7] ReactOS Project. *ntoskrnl/ke/thrdschd.c Source File*. ReactOS 0.4.17-dev. 2026. URL: https://doxygen.reactos.org/db/d73/thrdschd_8c_source.html (term. wiz. 11. 10. 2026).

[8] David A. Solomon i Mark E. Russinovich. “Thread Scheduling”. W: *Inside Microsoft Windows 2000*. 3 wyd. Microsoft Press, 2000. ISBN: 0735610215. URL: https://flylib.com/books/en/3.169.1.54/1/ (term. wiz. 11. 10. 2026).

Wykaz symboli i skrótów

ω	prędkość kątowa [rad/s]
M	moment obrotowy [Nm]
U	napięcie zasilania [V]
I	natężenie prądu [A]
θ	kąt obrotu wału silnika [$^\circ$ lub rad]
F	siła ciągu/nacisku [N]
k_p, k_i, k_d	nastawy (wzmocnienia) regulatora PID

ADC	(ang. *Analog-to-Digital Converter*) przetwornik analogowo-cyfrowy
DOF	(ang. *Degrees of Freedom*) stopnie swobody
IoT	(ang. *Internet of Things*) internet rzeczy
PLC	(ang. *Programmable Logic Controller*) programowalny sterownik logiczny

Spis rysunków

1 Wyniki analizy regresji liniowej oraz wykresy diagnostyczne reszt modelu 21
2 Struktura sieci neuronowej z dwiema warstwami ukrytymi 22
3 Schemat blokowy zamkniętego układu regulacji automatycznej z ujemnym sprzężeniem zwrotnym 23
4 Quadrocopter w trakcie lotu 24
5 Wizualizacja koncepcyjna quadrocoptera w trakcie lotu 24

Spis tabel

1 Zestawienie wyników pomiarów dla badanych punktów pracy 18
2 Porównanie parametrów technicznych wybranych platform mikrokontrolerowych 18
3 Wyniki pomiarów sprawności napędu 19

Spis załączników

1. Analiza błędów – pełny skrypt źródłowy zawierający zaimplementowane funkcje testowe oraz procedurę numerycznej analizy błędów metody trapezów.
2. Specyfikacja techniczna stacji roboczej wykorzystanej w pracy.
3. Dodatkowe zdjęcia stanowiska badawczego.

Załącznik 1
Analiza błędów

Zarządzenie Rektora PW [**rektorPW2022**] ściśle definiuje układ dokumentu, wskazując, że przedostatnim jego elementem jest „Spis załączników”, w którym wymienia się te materiały, które są fizycznie dołączone na samym końcu dokumentu pracy dyplomowej.

Poniżej znajduje się pełny skrypt zawierający funkcje testowe oraz procedurę analizy błędów.

Załącznik 2
Specyfikacja techniczna stacji roboczej wykorzystanej w pracy

Komponent	**Parametry techniczne / Model**
Procesor (CPU)	Intel Core i7-14700K (20 rdzeni, 28 wątków, taktowanie do 5.6 GHz)
Pamięć RAM	64 GB DDR5 Kingston Fury (2 $\times$ 32 GB, 6000 MHz, CL30)
Karta graficzna (GPU)	NVIDIA GeForce RTX 4080 Super (16 GB GDDR6X, 10240 rdzeni CUDA)
Dysk twardy	SSD Samsung 990 Pro 2 TB NVMe M.2 PCIe Gen4 (odczyt do 7450 MB/s)
Płyta główna	MSI MAG Z790 TOMAHAWK WIFI
Zasilacz	Corsair RM850x 850W (Certyfikat 80 Plus Gold)
System operacyjny	Linux Ubuntu 24.04 LTS / Windows 11 Pro (64-bit)
Środowisko i biblioteki	Python 3.12, CUDA Toolkit 12.4, PyTorch 2.3, MATLAB R2024a

Załącznik 3
Dodatkowe zdjęcia stanowiska badawczego

W niniejszym załączniku zgromadzono dodatkowe materiały graficzne, dokumentację fotograficzną stanowiska badawczego oraz szczegółowe wykresy przebiegów, które ze względu na swoje znaczne rozmiary i pomocniczy charakter nie zostały włączone bezpośrednio do zasadniczej części pracy. Zamieszczone poniżej materiały stanowią bezpośrednie uzupełnienie opisów eksperymentów zawartych w rozdziale 4 i umożliwiają pełniejszą, szczegółową weryfikację poprawności działania zaimplementowanego systemu.